

Python Programming — Take-Home Exam (Variant 1)

Question 1 - Movie Ratings (Lists and Strings)

Approach

- The movie data is first split into separate entries using commas.
- Each entry is separated into a movie title and rating using a colon.
- `strip()` removes extra spaces and the rating is converted to int.
- `average_rating()` collects all scores for the requested title and rounds the mean to one decimal place.
- `best_movie()` compares the averages of the distinct movie titles.
- `rating_counts()` stores the number of ratings for each movie in a dictionary.

Demo Output

```
Parsed ratings:
[('Dune', 8), ('Dune', 9), ('Barbie', 7),
 ('Dune', 10), ('Barbie', 9), ('Oppenheimer', 9),
 ('Barbie', 6)]
```

```
Average ratings:
Dune: 9.0
Barbie: 7.3
Oppenheimer: 9.0
Unknown: 0.0
```

```
Best movie:
Dune
```

```
Rating counts:
{'Dune': 3, 'Barbie': 3, 'Oppenheimer': 1}
```

Issues Faced / Checks

- Extra whitespace must not change the movie title.
- An unknown title must return 0.0 instead of causing division by zero.
- Dune and Oppenheimer tie at 9.0; Dune is returned because it is encountered first.

Question 2 - Gradebook (Dictionaries and Sets)

Approach

- The gradebook uses a dictionary of subjects containing student-grade dictionaries.
- `subjects_of()` adds matching subjects to a set.
- `takes_all()` uses set intersection to find students present in every subject.
- `student_average()` calculates the mean of all grades belonging to the requested student.
- `honor_roll()` keeps students whose average is at most the supplied limit and sorts the names alphabetically.

Demo Output

```
Subjects of anna: {'math', 'art'}  
Students taking all subjects: set()  
Average of clara: 1.15  
Average of unknown: 0.0  
Honor roll: ['clara']
```

Issues Faced / Checks

- Set intersection is used rather than counting subjects.
- Missing students return 0.0.
- The honor-roll boundary is inclusive because the condition is `<= limit`.
- The final list is alphabetically sorted.

Question 3 - Recipe Scaler (Functions)

Approach

- `scale_recipe()` accepts an arbitrary number of ingredient tuples with `*ingredients`.
- Every amount is multiplied by the requested serving count.
- The scaled amounts are returned in a dictionary and printed with the selected unit.
- Keyword arguments are displayed as cooking notes.
- Servings below 1 are rejected and the function returns 0.

Demo Output

```
Shopping list: Pancakes
Servings: 2
flour: 200g
milk: 300g
sugar: 40g
```

```
Shopping list: Fresh Juice
Servings: 3
orange juice: 600ml
water: 300ml
```

```
Shopping list: Chocolate Cake
Servings: 4
flour: 400g
sugar: 200g
butter: 160g
Cooking notes:
oven: 180C
time: 45min
```

```
Error: servings must be at least 1 for Invalid Recipe.
```

Issues Faced / Checks

- The ingredient count is variable, so `*ingredients` is required.
- The unit is configurable because one demonstration uses ml.
- `**options` allows several cooking notes.
- The invalid-serving case must return exactly 0.

Question 4 - Library (Classes and Inheritance)

Approach

- Book contains title, author, and year and implements `__str__`, `__eq__`, and `age()`.
- Equality compares title and author only.
- EBook inherits Book, adds `size_mb`, overrides `__str__`, and calculates download time.
- Library stores books in a list and ignores an equal book already present.
- `find_by_author()`, `oldest()`, and `__len__()` implement the remaining library operations.

Demo Output

```
Library length: 5
```

```
Books:
```

```
1984 by George Orwell (1949)
```

```
Dune by Frank Herbert (1965)
```

```
The Hobbit by J.R.R. Tolkien (1937)
```

```
Clean Code by Robert C. Martin (2008) - 5.0 MB
```

```
Python Crash Course by Eric Matthes (2019) - 8.0 MB
```

```
Book age:
```

```
77
```

```
Equality:
```

```
True
```

```
Books by George Orwell:
```

```
1984 by George Orwell (1949)
```

```
Oldest book:
```

```
The Hobbit by J.R.R. Tolkien (1937)
```

```
EBook download time:
```

```
4.0 seconds
```

Issues Faced / Checks

- A duplicate with the same title and author is ignored even if its year differs.
- EBook reuses the Book implementation through inheritance.
- Download time converts megabytes to megabits before dividing by network speed.
- The duplicate addition leaves the library length unchanged.

REPORT — QUESTION 5

Movie Review Extraction Pipeline (LLM + JSON)

Approach

For this question, I built a small pipeline that takes a movie review written in normal, informal language and asks a local language model to turn it into a simple JSON object. I used Qwen3-0.6B through LM Studio. The main idea was not to completely trust the model output, but to check it before using it in the Python program.

The program first sends the review to the model and asks for only three pieces of information: the movie title, the rating, and the sentiment. The response is then parsed as JSON. If the response is not valid JSON or does not follow the expected format, the program sends another request explaining what went wrong and asks the model to correct it.

1. Connection to LM Studio

The OpenAI-compatible Python client is connected to the local LM Studio server using the address `http://localhost:1234/v1`. The model name used in the program is `qwen3-0.6b`. For the normal pipeline, temperature is set to 0.0 so that the responses are as consistent as possible.

2. Sending the Review to the Model

The `ask_llm()` function is responsible for making the chat completion request. It sends a system instruction and the movie-review prompt. I use only `message.content` from the response because the assignment specifically says to ignore the model's separate `reasoning_content` field.

3. Building the Prompt

The `build_prompt()` function gives the model clear instructions about the required JSON format. It also tells the model not to add markdown, explanations, or code fences. This is important because extra text would make the response harder to process with `json.loads()`.

4. Parsing and Checking the Response

After receiving a response, the `parse()` function tries to load it as JSON. If that fails, it returns `None` instead of stopping the program. The `check_review()` function then checks whether the result has exactly the three required keys and whether each value is valid.

The rating check is slightly more careful because Python considers `True` and `False` to be integers in some type checks. The code therefore explicitly rejects boolean values and accepts only an actual integer from 1 to 10.

5. Retry Process

The retry mechanism is the main defensive part of the program. The model gets up to three attempts. If an answer fails validation, the reason for the failure is added to the next prompt. For example, if the rating is outside the allowed range, the next request tells the model that the rating is invalid and asks for corrected JSON.

If all attempts fail, the program prints an error message and returns `None`. The main pipeline skips failed results rather than crashing.

6. Reviews Used for the Demo

I used seven reviews with different writing styles. Some use a direct score such as 9 or 3, while others use informal expressions such as 'felt like a 6 maybe a 7' or describe the sentiment in words. The reviews include both positive and negative opinions so that the pipeline can test both sentiment values.

Demo Output

The program prints a small table containing the extracted title, rating, and sentiment. The exact model response can vary, so the final submitted report should contain the output from the actual run in LM Studio.

```

RESULTS
film review      | title      | rating | sentiment
-----
...              | Oppenheimer | 9      | positive
...              | Gladiator II | 7      | positive
...              | Barbie      | 8      | positive
...              | Killers of... | 5      | negative
...              | The Marvels | 3      | negative
...              | Aftersun    | 10     | positive
...              | Poor Things | 7      | positive

```

Problems Faced / Checks

One important issue with an LLM-based program is that the model does not always have to follow the requested format perfectly. Even when the information is correct, it could return additional text or an invalid JSON structure. This is why parsing and validation are both included.

Another point I had to consider was the rating type. A value such as 11 is an integer, but it is still invalid because the allowed range is only 1 to 10. Similarly, a boolean value must not be accepted as a rating.

Bonus — Temperature Comparison

For the bonus, the same reviews are run once with temperature 0.0 and once with temperature 1.5. The program counts how many reviews needed more than one attempt and how many failed completely.

Temperature	Reviews needing retries	Reviews failed
0.0	_____	_____
1.5	_____	_____

Observation

The temperature comparison is useful because it shows how much the model's output can change when randomness is increased. I would expect temperature 1.5 to be less consistent than temperature 0.0, which can result in more formatting or validation problems. The actual retry and failure counts should be filled in after running the program.

Real Transcript

The assignment specifically asks for one real transcript containing the prompt and the raw model reply. I have left these as placeholders here because they must come from the student's actual Qwen3-0.6B execution rather than being invented.

Prompt:

[PASTE THE ACTUAL PROMPT FROM YOUR RUN HERE]

Raw model reply (message.content):

[PASTE THE EXACT RAW MODEL REPLY FROM LM STUDIO HERE]

Conclusion

This question helped me understand that an LLM response should not be treated as automatically reliable just because the model was given a clear prompt. The program first asks for structured data, then parses and validates that data before using it. The retry step gives the model another opportunity to correct an invalid response. Overall, the pipeline provides a simple way of making free-text LLM output more suitable for a normal Python application.